

AEGIS

Android Execution, Ground-Truth Inspection & Security

PROJECT RESEARCH BLUEPRINT

An open-source, cross-platform toolkit for Android boot-chain reconnaissance, firmware metadata analysis, evidence collection and device-state comparison.

Initial target	Samsung Galaxy M11 (exact model to be confirmed)
Workstation	MacBook Air M4 / macOS
Academic context	Semester 3: OS, COA, DCN, DAA, Applied ML
Planning window	20–25 October 2026 intensive sprint; longer roadmap thereafter
Document status	Project proposal and execution plan — not a claim of completed implementation

CORE PRINCIPLE

> *Collect evidence first. State what is known. Label what is inferred. Never overclaim what the device does not expose.*

Contents

Contents 2

How to read this document . . . 3

1. Executive Summary 4

One-sentence definition . . . 4

Target outcome . . . 4

2. Problem Statement and Motivation 5

2.1 Problem . . . 5

2.2 Why this is a Core CS project . . . 5

2.3 Research problem . . . 5

2.4 Research questions . . . 5

3. Product Scope and Intended Users 6

3.1 Initial users . . . 6

3.2 Non-goals . . . 6

3.3 Product promise . . . 6

4. System Model: What AEGIS Investigates 7

4.1 Key concepts . . . 7

4.2 Evidence labels . . . 7

5. Proposed Architecture 8

5.1 Components . . . 8

5.2 Suggested stack . . . 8

5.3 Example command interface (proposed) . . . 9

6. Report Design: What a User Receives 10

6.1 Report sections . . . 10

6.2 Sample output (illustrative only) . . . 10

7. Cross-Disciplinary Semester 3 Mapping 11

7.1 What not to force . . . 11

7.2 Assessment / viva narrative . . . 11

8. Implementation Plan and Milestones 12

8.1 Repository layout (proposed) . . . 12

9. Gantt Chart: Intensive Sprint Until 25 October 13

10. Six-Week Roadmap After 25 October 14

10.1 Later research extensions . . . 14

11. Research Methodology and Evaluation 15

11.1 Method . . . 15

11.2 Evaluation metrics . . . 15

11.3 ML extension evaluation (optional) . . . 15

12. Safety, Privacy, Ethics and Threat Model 16

12.1 Safe-by-default rules . . . 16

12.2 Threat model . . . 16

13. Risks, Assumptions and Mitigations 17

13.1 Success criteria . . . 17

14. First-Day Runbook 18

14.1 On the Samsung phone . . . 18

14.2 On macOS . . . 18

14.3 Baseline data handling . . . 18

15. Deliverables and Final Demonstration 19

15.1 Demonstration script . . . 19

16. Open-Source Strategy and Project Positioning 20

16.1 Repository identity . . . 20

16.2 License . . . 20

16.3 Positioning statement . . . 20

16.4 CV positioning . . . 20

17. References and Further Reading 21

Glossary . . . 21

AEGIS PROJECT PRINCIPLE . . . 21

How to read this document

This blueprint describes the intended project, the first implementation scope, a six-day focused sprint through 25 October 2026, and a longer validation roadmap. Features are proposed until implemented and tested. The Samsung model number, Android version, bootloader state, and device-specific support remain unconfirmed at document creation.

1. Executive Summary

AEGIS is a proposed open-source toolkit that helps developers, students and mobile-systems researchers collect, normalize and interpret evidence about Android devices. Its first release will focus on read-only reconnaissance through Android Debug Bridge (ADB), a structured evidence model, a command-line interface, and machine-readable and human-readable reports. Deeper firmware parsing and device-specific boot-chain analysis will be added only where reliable sources and test evidence support them.

The initial experimental target is a Samsung Galaxy M11 connected by USB to a MacBook Air M4. The project is designed so the M11 is the first validation target rather than a hard-coded limit: generic collection capabilities will be separated from manufacturer-specific adapters.

The report is not intended to certify that a device is safe, detect all malware, or fully reverse-engineer proprietary bootloader code through ADB. Instead, it will document what was observed, where the evidence came from, what the result means, confidence/limitations, and what remains unknown.

One-sentence definition

> **AEGIS is an evidence-driven Android boot-chain and firmware reconnaissance toolkit that turns device observations into reproducible reports for learning, development and security research.**

Target outcome

- A cross-platform repository with documented setup and safe defaults.
- A CLI that checks ADB connectivity, collects a defined set of read-only properties and records errors explicitly.
- A versioned JSON schema and report generator (HTML first; PDF can follow).
- A boot-chain concept diagram with each relationship marked as documented, observed, inferred or unknown.
- Tests using mocked device outputs, plus a real baseline from the M11 once its exact model is confirmed.
- A research notebook/report explaining methodology, limitations, and reproducible findings.

2. Problem Statement and Motivation

2.1 Problem

Android device information is fragmented across shell commands, vendor documentation, firmware packages and community sources. Manual collection is inconsistent; outputs may be difficult to compare across devices; and users can mistakenly treat a reported property as definitive proof of device integrity. AEGIS aims to make collection repeatable and interpretation explicit.

2.2 Why this is a Core CS project

The project combines operating-system concepts, boot and hardware architecture, evidence collection, command-line engineering, graph modelling, network communication, testing and security reasoning. The technical challenge is not to print properties, but to design a robust tool that handles inconsistent device capabilities and clearly communicates the limits of its evidence.

2.3 Research problem

How can a cross-platform, read-only toolkit collect and normalize Android boot-related evidence across heterogeneous devices while preserving provenance, making uncertainty explicit, and enabling repeatable comparison between device snapshots?

2.4 Research questions

- RQ1: Which boot, kernel, build and partition-related attributes can be collected reliably through standard ADB access?
- RQ2: How should AEGIS represent unavailable, unsupported, contradictory or low-confidence evidence?
- RQ3: How consistently can a common report schema represent observations from different Android manufacturers and versions?
- RQ4: Can two snapshots be compared to surface meaningful configuration changes without misclassifying normal changes as security incidents?
- RQ5 (extension): Does a transparent rule-based baseline outperform or complement an ML anomaly model on a sufficiently sized, labelled dataset?

3. Product Scope and Intended Users

3.1 Initial users

Audience	Need / use case	AEGIS value
CS students and educators	Learn boot flow, kernel/userspace boundaries and Android internals using a real device.	Guided explanations, diagrams and reproducible exercises.
Android developers	Record device properties and debug environment consistently.	Repeatable baseline report and environment metadata.
Custom ROM / kernel developers	Establish device baseline and gather available compatibility evidence.	Capability map, source links and clearly stated unknowns.
Mobile security researchers	Compare observable boot/firmware configuration and preserve evidence provenance.	Evidence ledger, hashes where applicable, snapshot diff and export.
Repair / research labs	Document state before and after authorized maintenance.	Time-stamped reports and change history; not a forensic certification.

3.2 Non-goals

- Not a one-click root, bootloader unlocker, flashing utility or exploit framework.
- Not a universal bootloader decompiler and not guaranteed to access proprietary bootloader internals.
- Not a malware scanner, attestation provider, formal security certification or proof that a phone is uncompromised.
- Not an automatic firmware downloader or redistributor; firmware licensing and source authenticity must be checked.
- Not an ML product at MVP stage. ML is optional and gated on enough labelled data.

3.3 Product promise

AEGIS will state what it can support, what evidence it collected, and what it cannot establish. A capability can be 'supported', 'unsupported', 'inaccessible', 'not applicable' or 'unknown'; absence of evidence must never be presented as evidence of absence.

4. System Model: What AEGIS Investigates

The diagram below is a generic conceptual boot model derived from Android documentation. It is not yet a verified map of the Galaxy M11. The exact bootloader stages, partition names and hand-off mechanisms vary by manufacturer, chipset, Android release and device design. [1][2][3]



Conceptual reference flow; exact stages and partitions vary by device and Android version.

4.1 Key concepts

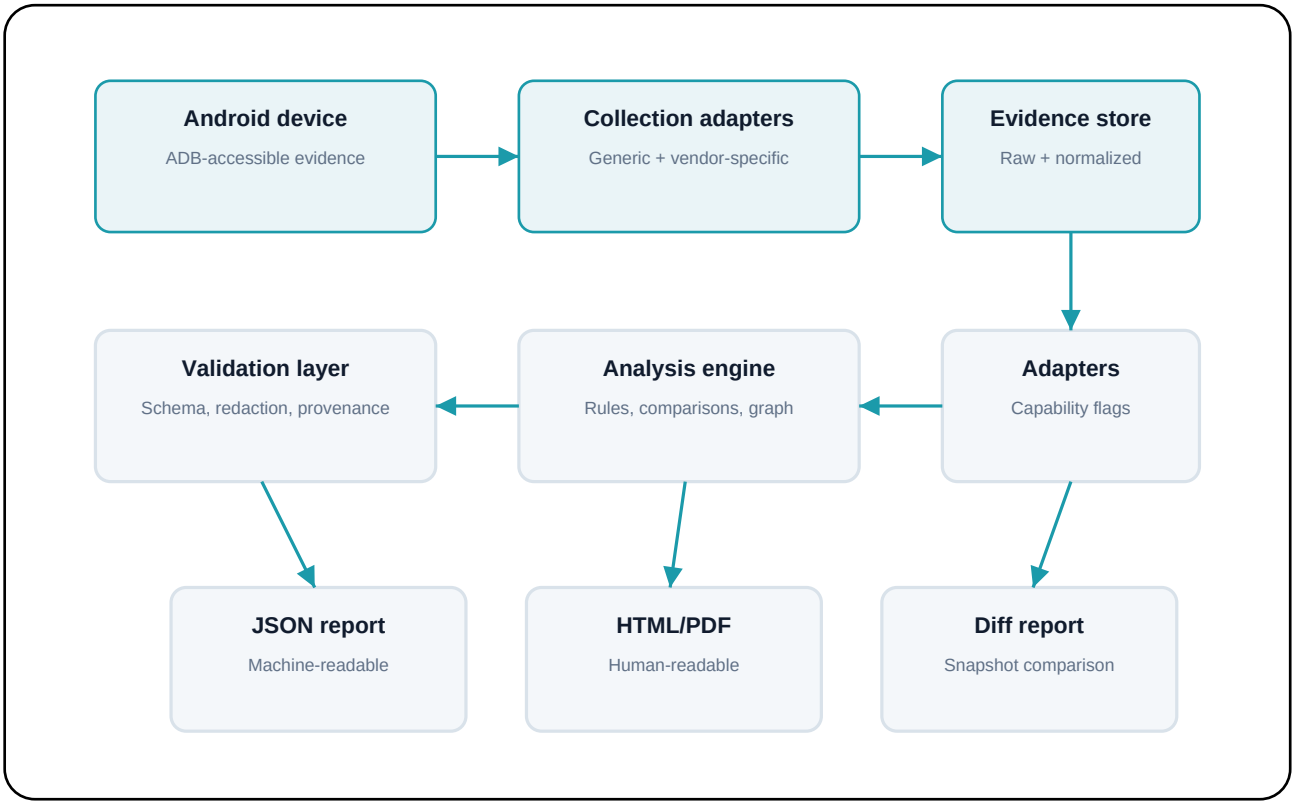
Concept	Plain-language explanation	What AEGIS can do
Bootloader	Early software that initializes the platform, checks boot state and loads the kernel.	Collect visible boot-state clues; document stages from trusted technical sources. Direct bootloader internals may remain inaccessible.
Kernel	The core of the operating system that manages CPU, memory, drivers and processes.	Record kernel release and exposed command-line metadata where available.
Partitions	Logical storage regions holding different system or data components.	Inspect accessible partition listings; do not assume /proc/partitions is a complete physical map.
Verified Boot / AVB	A chain of checks intended to verify integrity/authenticity of booted software and support rollback protection.	Record available reported states and compare with documented semantics; cannot independently prove the entire chain from a property alone.
Userspace / init	After kernel startup, Android starts init, mounts required filesystems and launches services.	Document the expected flow; collect observable OS/build evidence, not every internal event.

4.2 Evidence labels

- **Observed:** directly returned by a command or calculated from a collected file.
- **Derived:** computed deterministically from observed values (for example, SHA-256 of a local file).
- **Documented:** supported by an identified technical reference for a stated device/version scope.
- **Inferred:** a reasoned conclusion that is not directly observed; assumptions must be recorded.
- **Unknown / inaccessible:** data could not be established through the available method.

5. Proposed Architecture

The architecture separates device communication from analysis and presentation. That separation is the main design choice that lets AEGIS grow from one Samsung test device into a multi-device toolkit.



5.1 Components

Component	Responsibility	MVP?
CLI / orchestration	Commands, arguments, exit codes, help, output directory and logging.	Yes
ADB transport	Check adb availability, enumerate devices, confirm authorization, run allow-listed read-only commands with timeouts.	Yes
Collectors	Collect properties, kernel version, build fingerprint, patch level, command line and accessible partition metadata.	Yes
Evidence schema	Store key, value, source command, timestamp, collection status, privacy classification and evidence label.	Yes
Normalizer / validator	Normalize values without silently changing raw output; validate schema and flag missing/contradictory fields.	Yes
Analysis engine	Rule-based interpretation, capability status and snapshot comparison.	MVP-lite
Report renderers	JSON first; HTML report and diagrams after schema stabilizes.	JSON yes
Vendor adapters	Optional device-family logic based on verified sources and tested behaviour.	Later
ML extension	Experimental anomaly detection with baseline comparisons and documented limitations.	No

5.2 Suggested stack

- **Primary language:** Python 3.11+ for rapid iteration, subprocess handling, schemas, testing and report generation.
- **Device interface:** Android Debug Bridge (ADB), installed via Android platform-tools.

- **Data:** JSON with a versioned schema; optional SQLite later if history becomes useful.
- **Validation:** pytest plus schema validation (for example, JSON Schema or Pydantic).
- **Reports:** HTML first; PDF export can be added once the data model is stable.
- **Diagrams:** generated from normalized evidence and documented relationships; no hard-coded claim that every device follows identical details.
- **Packaging:** Python package/CLI and pinned development dependencies; cross-platform CI where practical.

5.3 Example command interface (proposed)

```
aegis doctor
aegis devices
aegis collect --device <serial> --out ./reports/run-001
aegis report ./reports/run-001/evidence.json --format html
aegis diff ./reports/baseline.json ./reports/after-update.json
```

These commands are proposed interface examples, not existing functionality.

6. Report Design: What a User Receives

6.1 Report sections

Section	Contents	Why it matters
Executive summary	Device identity summary, collection time, overall completeness and limitations.	A reader can quickly understand what was and was not checked.
Device profile	Manufacturer/model strings, Android release, build fingerprint, security patch level and kernel version where available.	Reproducible identification of the observed software environment.
Boot/security evidence	Relevant boot properties, source command, raw value, normalized interpretation and confidence label.	Avoids turning a property into an unsupported security verdict.
Storage/partition observations	Accessible block-device metadata and caveats about visibility.	Helps learners understand the limits of user-space views.
Evidence ledger	Command, timestamp, status, raw output reference, tool version and redaction notes.	Supports repeatability and auditing.
Changes / diff	Added, removed and changed fields between two snapshots.	Makes update investigation more efficient.
Unknowns / recommendations	Unsupported checks, likely reasons, safe next steps and reference links.	Makes limitations useful rather than hiding them.

6.2 Sample output (illustrative only)

Check	Result	Interpretation
Build fingerprint	Collected	Raw value retained; helps identify the observed build.
Boot-state property	Observed / absent / inaccessible	If present, interpret against documentation; not independent proof of integrity.
Full bootloader code	Not inspected by ADB-only MVP	Requires separate device-specific evidence and may remain proprietary.
Firmware authenticity	Not established by metadata alone	Requires trusted source/signature verification appropriate to the package.

AEGIS must not output a single green 'secure' badge based only on properties. Prefer per-check states such as PASS (specific test), OBSERVED, NOT TESTED, UNKNOWN, INACCESSIBLE and NEEDS REVIEW. A PASS applies only to the stated test and evidence scope.

7. Cross-Disciplinary Semester 3 Mapping

Course	Concepts used	AEGIS implementation / deliverable
Operating Systems (OS)	Kernel vs userspace, process startup, init, mounts, permissions, filesystems, system interfaces.	Boot-flow explanation, kernel metadata collector, command execution and access-limit analysis.
Computer Organization & Architecture (COA)	Reset/startup sequence, CPU and memory initialization, firmware, hardware-software boundary.	Conceptual boot model and device-specific research note distinguishing documented hardware flow from observed OS data.
Data Communication & Networks (DCN)	USB as transport, host-device communication, structured data transfer, optional local report serving.	ADB connection diagnostics; optional local-only report dashboard in a later phase.
Design & Analysis of Algorithms (DAA)	Graph representation, traversal, dependency ordering, hashing, diff complexity.	Boot dependency graph with provenance labels; snapshot diff and complexity discussion.
Application of ML	Feature engineering, anomaly detection, train/test split, precision/recall, false positives.	Optional research extension only after collecting enough labelled, comparable firmware metadata.

7.1 What not to force

AEGIS does not need machine learning to be a strong systems project. A small dataset with no reliable labels can make ML claims misleading. The academically sound approach is to complete the deterministic evidence pipeline first, then ask whether a well-defined ML experiment adds measurable value.

7.2 Assessment / viva narrative

The project can be explained as an integration of OS and COA fundamentals, a practical ADB/USB interface, algorithmic graph and diff operations, and optional ML evaluation. The strongest viva evidence will be a live demonstration, a known-good test fixture, a failure case, and an explanation of why some information cannot be obtained from an unprivileged Android shell.

8. Implementation Plan and Milestones

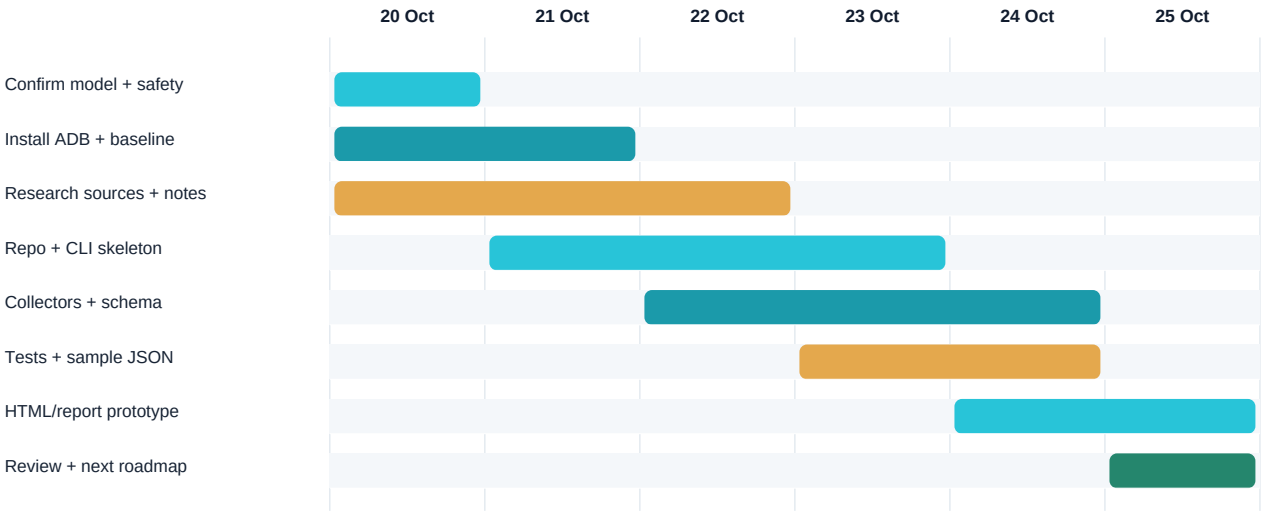
Milestone	Tasks	Acceptance evidence
M0 — Environment	Confirm model; install platform-tools; authorize ADB; capture baseline safely.	Device is detected; baseline files saved; no device modification.
M1 — CLI skeleton	Project structure, command parser, logging, error model, `doctor` and `devices`.	Commands run with useful messages on connected, absent and unauthorized device cases.
M2 — Collectors	Allow-listed read-only collectors and raw output preservation.	Fixture tests pass; collection records source and status.
M3 — Evidence schema	Versioned JSON schema, timestamps, privacy tags, provenance and validation.	Schema validates good fixture and rejects malformed fixture.
M4 — First report	Human-readable summary, capability states, unknowns and caveats.	Report generated from fixture and real baseline; claims trace to evidence.
M5 — Diff engine	Compare two snapshots, normalize stable fields, classify changes without threat claims.	Unit tests cover additions, removals, value changes and missing values.
M6 — Research validation	Test on M11 and, when available, a second device; document differences.	Compatibility matrix marks tested vs untested models honestly.
M7 — Public release	README, license, threat model, privacy guidance, sample redacted reports, CI.	A new user can follow install instructions and reproduce a sample run.

8.1 Repository layout (proposed)

```
aegis/
■■■ pyproject.toml
■■■ README.md
■■■ LICENSE
■■■ docs/
■ ■■■ architecture.md
■ ■■■ threat-model.md
■ ■■■ device-support.md
■■■ src/aegis/
■ ■■■ cli.py
■ ■■■ adb.py
■ ■■■ collectors/
■ ■■■ schema.py
■ ■■■ normalize.py
■ ■■■ analysis/
■ ■■■ reports/
■ ■■■ adapters/
■■■ tests/
■■■ fixtures/
■■■ test_collectors.py
```

9. Gantt Chart: Intensive Sprint Until 25 October

Planning assumption: the current date is 20 October 2026 and the user is available through 25 October inclusive. This is a focused six-day sprint, not a promise to complete full cross-device firmware reverse engineering. The sprint target is a safe baseline, a working MVP skeleton, initial tests and a clear technical report.



Date	Focus	Concrete output by end of day
Tue 20 Oct	Project setup + confirm prerequisites	Confirm exact model; create repo; install/verify ADB; safe-device checklist.
Wed 21 Oct	Baseline collection + source research	Raw baseline bundle; annotated list of observable properties; initial source register.
Thu 22 Oct	CLI skeleton + data model	`doctor` and `devices` command stubs; JSON schema v0.1; logging/errors.
Fri 23 Oct	Collectors + tests	Read-only collectors; mocked fixtures; tests for missing/unauthorized device.
Sat 24 Oct	Report and diff prototype	First HTML/Markdown report; simple snapshot comparison; caveat labels.
Sun 25 Oct	Validation + documentation	Demo walkthrough; README; limitations; backlog and next six-week plan.

Sprint exit criteria: a reproducible baseline, a runnable but appropriately scoped prototype, tests for the key failure cases, and a documented plan. If implementation starts from zero, deeper firmware parsing and multiple-device validation remain post-sprint work.

10. Six-Week Roadmap After 25 October

Week	Focus	Deliverables
Week 1	Harden MVP	Stable CLI, schema v1, structured errors, test fixtures and CI.
Week 2	Report engine	HTML report, privacy redaction, source/evidence links and export validation.
Week 3	Device research	Exact M11 variant research; vendor/kernel sources; documented boot-chain map with evidence labels.
Week 4	Snapshot diff	Comparison rules, noise filtering, report change summaries and tests.
Week 5	Cross-device pilot	Test on a second device if available; capability matrix; adapter contract.
Week 6	Release candidate	README, architecture docs, threat model, sample report, license, release checklist.

10.1 Later research extensions

- Firmware package analysis: only after exact model identification and trusted package sourcing; parse metadata and hashes before attempting deeper binary work.
- Boot image / device-tree analysis: use matching public sources and safe copies of firmware; document toolchain and version assumptions.
- Interactive boot-chain graph: distinguish documented generic stages from device-specific verified relationships.
- Performance / resource benchmarks: optional and separate from the security report; control background processes and repeat runs.
- ML anomaly detection: only after defining a dataset, labels, baseline and evaluation metrics; compare against simple rules.
- Additional vendor adapters: implement only when a device can be tested or the evidence source is strong enough to state limitations.

11. Research Methodology and Evaluation

11.1 Method

- Define a narrow collection protocol with allow-listed, read-only commands and explicit timeouts.
- Preserve raw outputs separately from normalized values; record command, timestamp, tool version, exit status and error details.
- Use a versioned schema and deterministic normalization rules.
- Validate collectors first against fixtures, then against the M11 baseline.
- Compare results against official Android documentation and device-specific sources where available.
- Repeat collection in the same environment to identify unstable fields and nondeterministic output.
- For cross-device validation, record model, Android release, build fingerprint and access conditions for each device.
- Publish redacted sample reports and enough instructions for another person to reproduce the workflow.

11.2 Evaluation metrics

Metric	Definition / target approach
Collection coverage	Fraction of planned checks with a clear status (collected, unsupported, inaccessible, error or not applicable); do not equate coverage with security.
Repeatability	Compare repeated runs on the same unchanged device; identify stable and expected-to-vary fields.
Schema validity	All valid fixtures validate; malformed fixtures produce clear failures.
Error clarity	Disconnected, unauthorized, missing ADB and restricted command cases produce actionable messages.
Diff correctness	Known synthetic changes are detected; missing fields are not misreported as removals without context.
Portability	Run automated tests on macOS and at least one additional OS in CI; actual device validation is reported separately.
Evidence traceability	Every interpreted claim links to an observed value or an explicit external reference.

11.3 ML extension evaluation (optional)

If AEGIS Research later includes anomaly detection, define a concrete prediction task first. Candidate task: flag metadata snapshots that differ from a baseline in a way worth human review. Use legitimate, versioned samples and avoid treating a version change as malicious by default. Compare a rule-based baseline to an ML model using precision, recall, false-positive rate and a held-out test split. If labelled data is too small or unrepresentative, report that ML is not justified.

12. Safety, Privacy, Ethics and Threat Model

12.1 Safe-by-default rules

- MVP commands are read-only and allow-listed; no flash, erase, unlock, root or reboot commands.
- Never instruct users to unlock the bootloader as a prerequisite for baseline collection; unlocking can wipe data and affect device security.
- Do not collect personal files, messages, photos, app databases, tokens or account content.
- Treat device serials, IMEIs, Wi-Fi identifiers and other unique values as sensitive; redact them by default in shareable reports.
- Do not upload evidence automatically. Local-first storage and explicit export are the default.
- Do not claim firmware authenticity from a hash alone; a hash only identifies bytes unless compared with a trusted reference.
- Do not label a device compromised based only on a missing property, unusual build string or one configuration difference.
- Keep vendor firmware files local unless their redistribution terms clearly permit sharing.

12.2 Threat model

Risk	Impact	Mitigation
Overclaiming security	Users infer a device is safe/unsafe from weak evidence.	Per-check status, evidence provenance, confidence and explicit limitations.
Sensitive identifiers in report	Privacy leak when reports are posted publicly.	Redaction defaults, preview before export, synthetic examples.
Malicious / malformed device output	Parser crash or misleading report.	Treat output as untrusted input; validate, escape HTML and limit output size.
Command injection	Unintended host/device command execution.	No shell interpolation for user input; allow-list commands; subprocess argument arrays and timeouts.
Firmware source uncertainty	Wrong package or unsafe conclusions.	Record source URL, model/region/build match and signature/hash checks where possible.
Device damage / data loss	Phone may become unusable or lose data.	No destructive commands in MVP; backup and recovery guidance for any future experimental phase.

13. Risks, Assumptions and Mitigations

Risk / assumption	Why it matters	Mitigation / decision
Exact M11 model not confirmed	Samsung model variants can differ in chipset, firmware and support.	Confirm Settings → About phone → Model number before device-specific claims.
ADB exposes limited information	Some bootloader and partition details are not available without elevated access.	Treat access limits as expected; support external references and optional offline firmware analysis later.
Cross-platform behaviour differs	ADB installation, permissions and shell output vary by host OS/device.	Abstract transport, use fixtures and CI; test on each supported OS.
Scope creep	OS porting, ML, interactive UI and full firmware reverse engineering are large projects.	Deliver a narrow MVP first; keep all advanced features as explicit later milestones.
Insufficient ML dataset	Anomaly detection may be unvalidated or produce false positives.	Do not include ML in MVP; require labelled data and baseline comparison before proceeding.
Availability ends 25 Oct	Only six days for initial sprint.	Target setup, baseline, skeleton, tests and report prototype—not complete research-grade multi-device support.

13.1 Success criteria

- A fresh user can install prerequisites and understand setup from the README.
- AEGIS detects common connection states without crashing or running destructive commands.
- The output JSON is versioned, valid and traceable to raw evidence.
- A report clearly separates observation, interpretation, documentation, inference and unknowns.
- Automated tests cover normal and failure paths.
- At least one actual device baseline is collected and redacted for publication.
- The compatibility matrix does not claim support for untested models.

14. First-Day Runbook

14.1 On the Samsung phone

- Open Settings → About phone → Software information; record model number, Android version, build number and security patch level.
- Enable Developer options by tapping Build number seven times if necessary.
- Enable USB debugging, connect by data-capable USB cable and accept the RSA authorization prompt.
- Do not enable OEM unlocking, unlock the bootloader, enter Download Mode, root or flash anything for this baseline.

14.2 On macOS

```
brew install android-platform-tools
adb version
adb devices -l
mkdir -p ~/M11-BootChain
cd ~/M11-BootChain
adb shell getprop > device-properties.txt
adb shell uname -a > kernel.txt
adb shell cat /proc/cmdline > kernel-cmdline.txt
adb shell cat /proc/partitions > partitions.txt
adb shell getprop ro.build.fingerprint > build-fingerprint.txt
adb shell getprop ro.build.version.security_patch > security-patch.txt
adb shell getprop | grep -Ei 'ro.boot|verifiedboot|vbmeta|verity|secureboot|bootloader|flash.locked' >
boot-properties.txt
```

These commands collect initial evidence through ordinary ADB access. Some values may be absent or restricted. `/proc/partitions` is not a complete physical partition map, and `getprop` values are reports from the running software, not independent proof of hardware trust.

14.3 Baseline data handling

- Keep the raw baseline local and read-only; do not edit it after collection.
- Create a copy for redaction and sharing; do not publish serial numbers or unique identifiers.
- Record macOS version, ADB version, collection time and phone build.
- If a command fails, retain the exit status/error and mark the field inaccessible or failed rather than substituting a guess.

15. Deliverables and Final Demonstration

Deliverable	Definition of done
Public repository	License, setup guide, project scope, supported platforms and safe-use statement.
AEGIS CLI	Connection checks and read-only collection commands with clear exit codes.
Evidence schema	Versioned JSON with raw/normalized values, source, timestamp, status and privacy metadata.
Report generator	Human-readable report with observed evidence, limitations, unknowns and source references.
Snapshot diff	Compare two saved reports and identify field-level changes without automatic threat labels.
Test suite	Mock fixtures for connected, unauthorized, absent device, missing property and malformed output cases.
Research note	Methodology, conceptual boot-chain diagram, results from actual testing, limitations and future work.
Demo	Connect device → collect → view report → compare snapshots → explain an unknown/limitation.

15.1 Demonstration script

- Show the device model and confirm ADB authorization.
- Run AEGIS preflight checks and explain the required permissions.
- Collect a baseline and open the JSON evidence bundle.
- Generate a report and trace one interpretation back to its source value.
- Compare against a second synthetic or real snapshot and explain a detected change.
- Show a check marked unknown/inaccessible and explain why the tool does not guess.
- Finish with limitations, tested-device matrix and future research plan.

16. Open-Source Strategy and Project Positioning

16.1 Repository identity

Name: AEGIS
Expansion: Android Execution, Ground-Truth Inspection & Security
Tagline: An open-source Android boot-chain and firmware intelligence toolkit.
Initial target: Samsung Galaxy M11, exact model to be confirmed.
Design principle: Evidence first; uncertainty is a valid result.

16.2 License

MIT and Apache-2.0 are both plausible choices for a general-purpose open-source toolkit. Apache-2.0 includes explicit patent-related terms; MIT is shorter and permissive. Choose after reviewing dependencies, intended contribution model and institutional guidance. The project license does not override the licenses or distribution restrictions of vendor firmware and third-party assets.

16.3 Positioning statement

> *AEGIS helps Android learners, developers and researchers collect and compare device boot-related evidence in a repeatable, transparent way—without pretending that unprivileged inspection can reveal every proprietary implementation detail.*

16.4 CV positioning

Use present-progressive wording while it is under development. After implementation, replace planned capabilities with verified achievements and measurable results. A defensible CV entry should mention the actual language, supported device(s), tests, report format and concrete technical finding—never claim universal device support or complete bootloader reverse engineering unless proven.

17. References and Further Reading

[1] Android Open Source Project (AOSP), “Verified Boot.”
https://source.android.com/docs/security/features/verifiedboot
Background on the chain of trust, boot state and rollback protection.

[2] Android Open Source Project (AOSP), “Boot flow.”
https://source.android.com/docs/security/features/verifiedboot/boot-flow
Describes boot flow and how boot state is communicated to Android.

[3] Android Open Source Project (AOSP), “Bootloader overview.”
https://source.android.com/docs/core/architecture/bootloader
Explains bootloader responsibilities and boot partition verification.

[4] Android Open Source Project (AOSP), “Partitions overview.”
https://source.android.com/docs/core/architecture/partitions
Explains Android system and vendor partition architecture.

[5] Android Developers, “Android Debug Bridge (adb).”
https://developer.android.com/tools/adb
Official documentation for the ADB client-server-device tool.

[6] Android Open Source Project (AOSP), “Use Verified Boot.”
https://source.android.com/docs/security/features/verifiedboot/verified-boot
Details on verified partitions and verification behaviour.

Reference note: official Android documentation describes platform concepts, not necessarily the exact Samsung Galaxy M11 implementation. Device-specific claims require the exact model/region/build and suitable device-specific evidence. Links were consulted for planning on 20 October 2026; readers should check current documentation as the platform evolves.

Glossary

Term	Meaning
ADB	Android Debug Bridge; host tool for communicating with an Android device.
AVB	Android Verified Boot; Android's verified-boot reference implementation and related mechanisms.
Boot chain	Sequence of boot stages that establish execution from early firmware to the operating system.
Kernel	Core OS component managing hardware resources and providing services to userspace.
Provenance	Record of where evidence came from and how it was collected or transformed.
Snapshot	A time-bounded collection of device observations used for later comparison.
GSI	Generic System Image; generic Android system image, not a universal image guaranteed to work on every device.

AEGIS PROJECT PRINCIPLE

A reliable tool is not the one that claims to know everything. It is the one that shows its evidence, exposes its uncertainty, and lets another person reproduce the result.